

FraudGuard: Fake Job Listing Detection using Deep Learning and Agentic AI



Data Science and AI Lab Project – Final Presentation

Group: Group 9

Arun Dutta · Hritik Roshan Maurya · Vivek Bajaj · Vishwas Mehta

Date: April 2026

The Problem: Fake Job Listings at Scale



3-5% of all online job postings are fraudulent.

DATA METRIC



Modern fraud postings mimic legitimate ads with linguistic sophistication — keyword filters fail.



Victims lose money, personal data, and time — real-world impact on Indian job seekers.

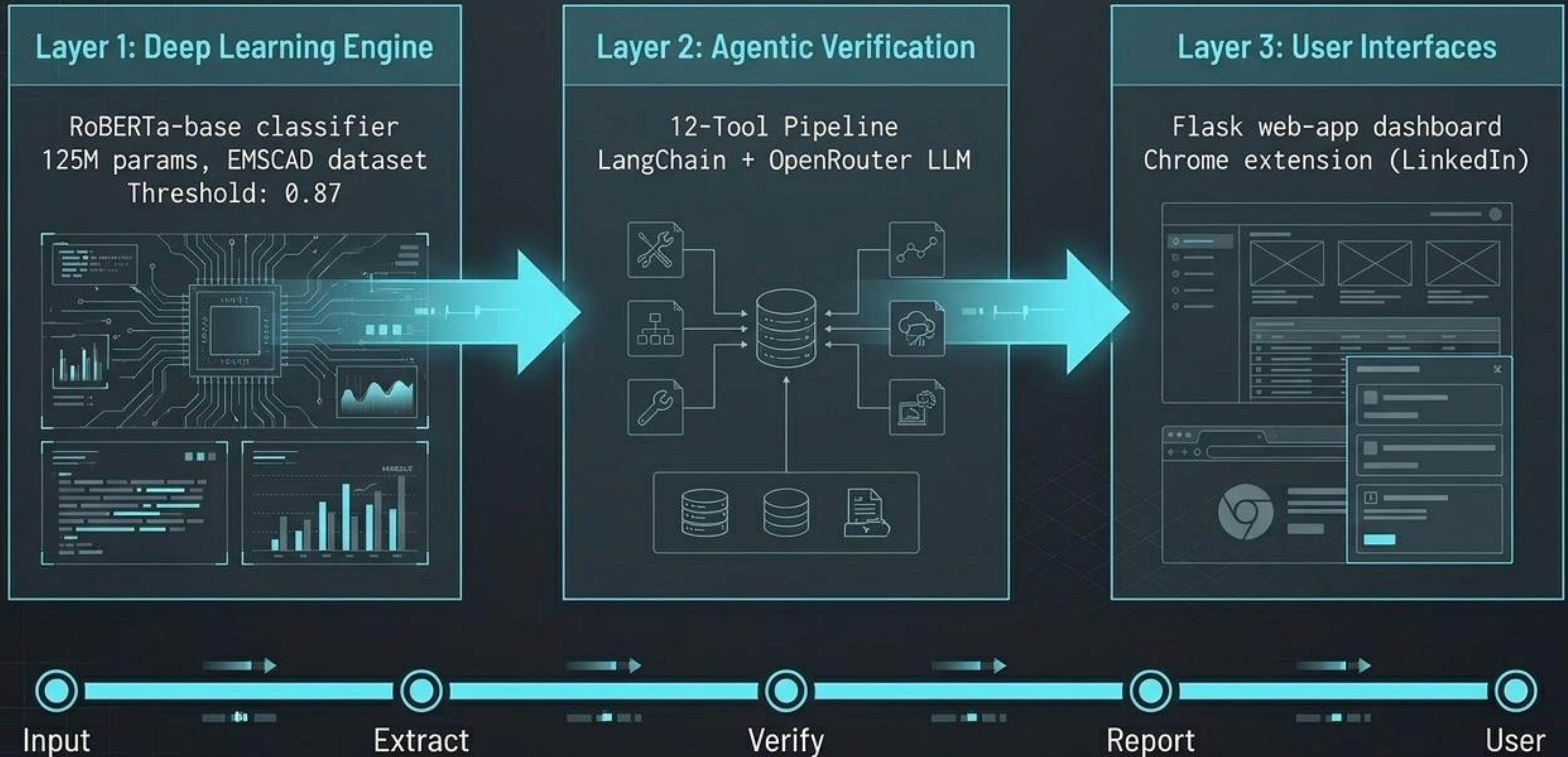


No existing tool combines deep NLP + external verification + human-readable explanation.



FraudGuard doesn't just output a probability score;
it **cross-references** the web to prove its verdict.

FraudGuard – Three-Layer Architecture



EMSCAD Dataset — 17,880 Job Postings

17,880 samples, 4.84% fraudulent (866 fraud, 17,014 legitimate) — 20:1 imbalance.

18 features: 5 free-text, 9 structured metadata, 3 binary.

After deduplication: 15,787 unique samples.

Split: 70% train / 15% validation / 15% test (stratified).

Preprocessing Innovation: The [SEP] Strategy

Location: New York [SEP] Salary: 500-1000 [SEP] title text [SEP]

description...

! 512-Token
Truncation Limit



Structured metadata fields are placed FIRST in the [SEP]-joined input string.
This protects critical signals from 512-token truncation.

Module: Input Sequence Construction

```
STRUCT_COLS = ['location', 'department', 'salary_range',  
'employment_type', 'required_experience', 'required_education',  
'industry', 'function', 'has_company_logo']  
TEXT_COLS = ['title', 'description', 'requirements',  
'company_profile', 'benefits']
```

```
def build_input_text(row):  
    parts = []  
    for col in STRUCT_COLS:  
        val = str(row.get(col, '')) or ''  
        if val and val.lower() not in ('nan', 'none', ''):  
            parts.append(f"{col.replace('_', ' ').title()}:  
{val}")  
    for col in TEXT_COLS:  
        val = str(row.get(col, '')) or ''  
        if val and val.lower() not in ('nan', 'none', ''):  
            parts.append(val)
```

```
[return ' [SEP] '.join(parts)]
```

```
df['input_text'] = df.apply(build_input_text, axis=1)  
df['label'] = df['fraudulent'].astype(int)
```

Module: RoBERTa Tokenization

```
# Module: RoBERTa Tokenization
```

```
from transformers import AutoTokenizer
```

```
MODEL_NAME = 'roberta-base'
```

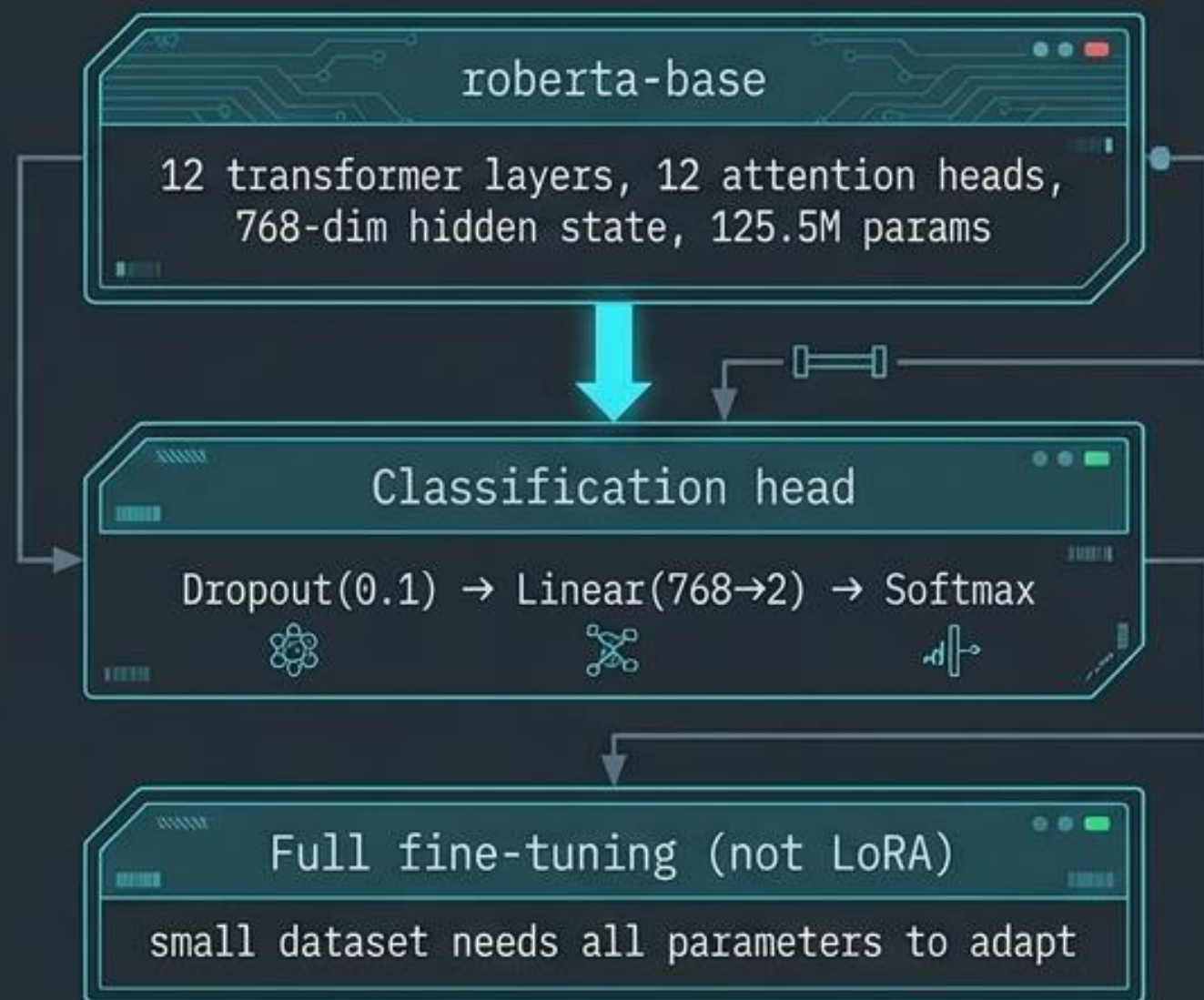
```
MAX_SEQ_LEN = 512
```

```
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
```

```
def tokenize_fn(batch):  
    return tokenizer(batch['input_text'],  
                      truncation=True,  
                      padding='max_length',  
                      max_length=MAX_SEQ_LEN)
```


RoBERTa-base — Why and How

Architecture



The Challenge: 20:1 Class Imbalance



25-Trial Optuna HPO — Systematic Search with Hard Floors

lr: 2.59e-5	warmup: 0.1506	batch: 16	weight_decay: 0.0702
focal_gamma: 1.6920	fraud_class_weight: 2.8251		epochs: 9 (stopped at 7)

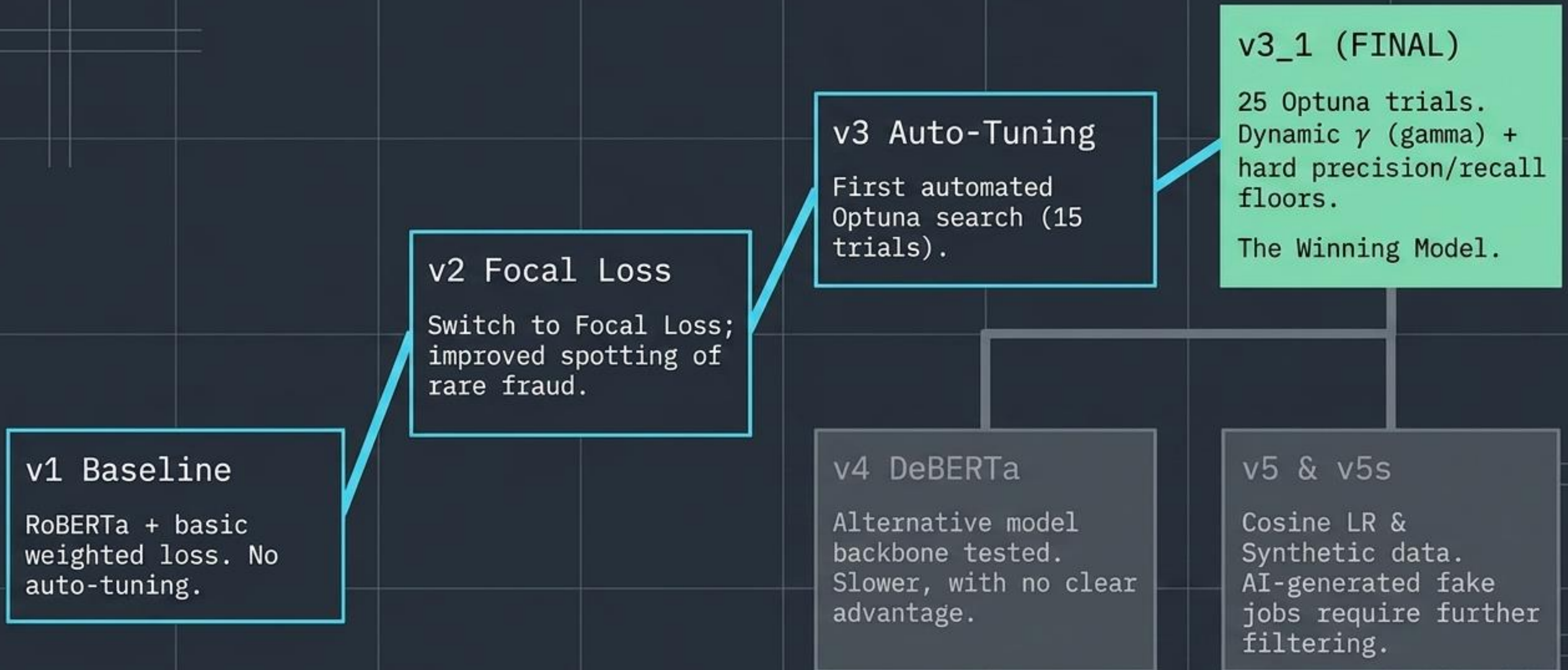
Key Innovation:

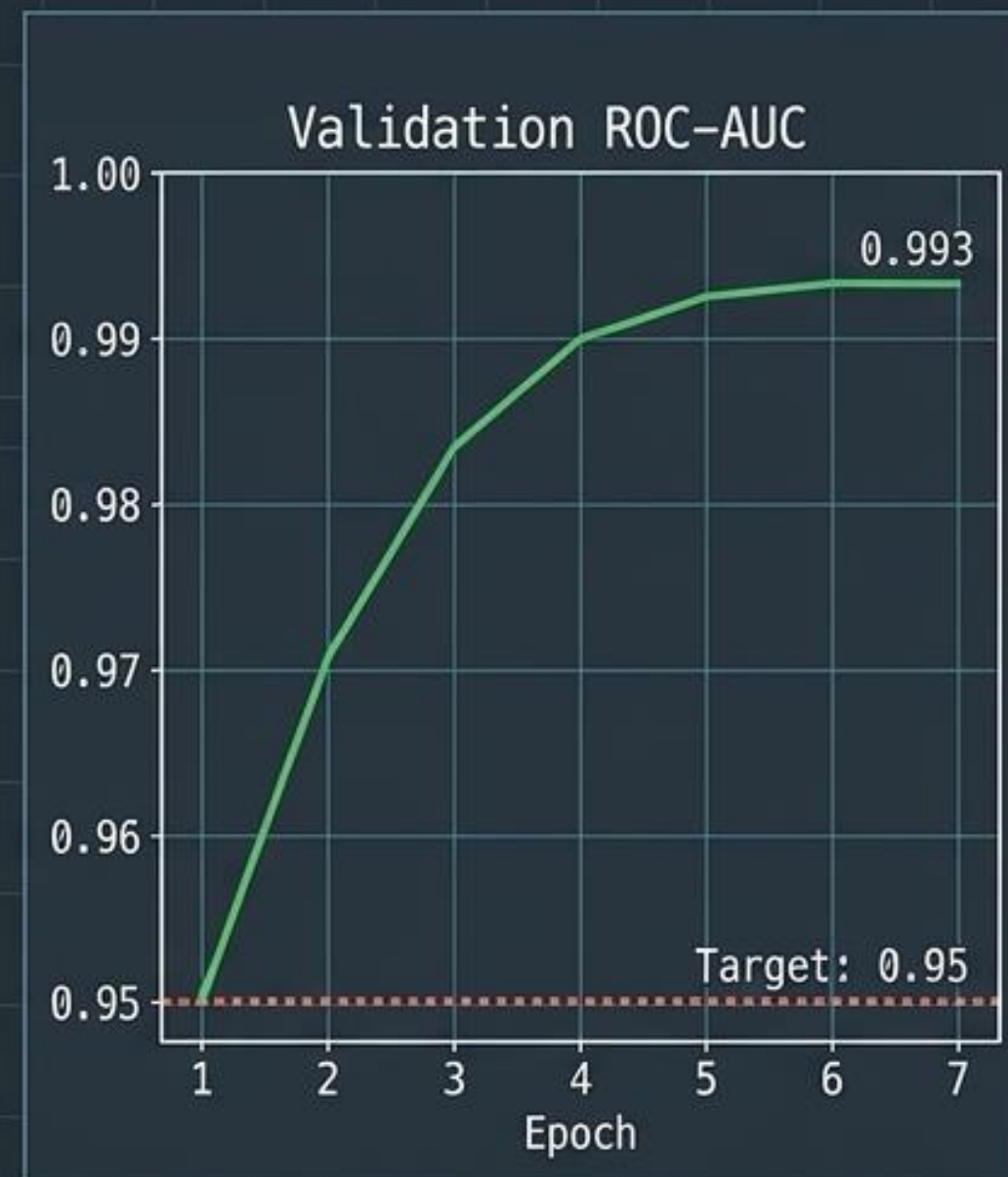
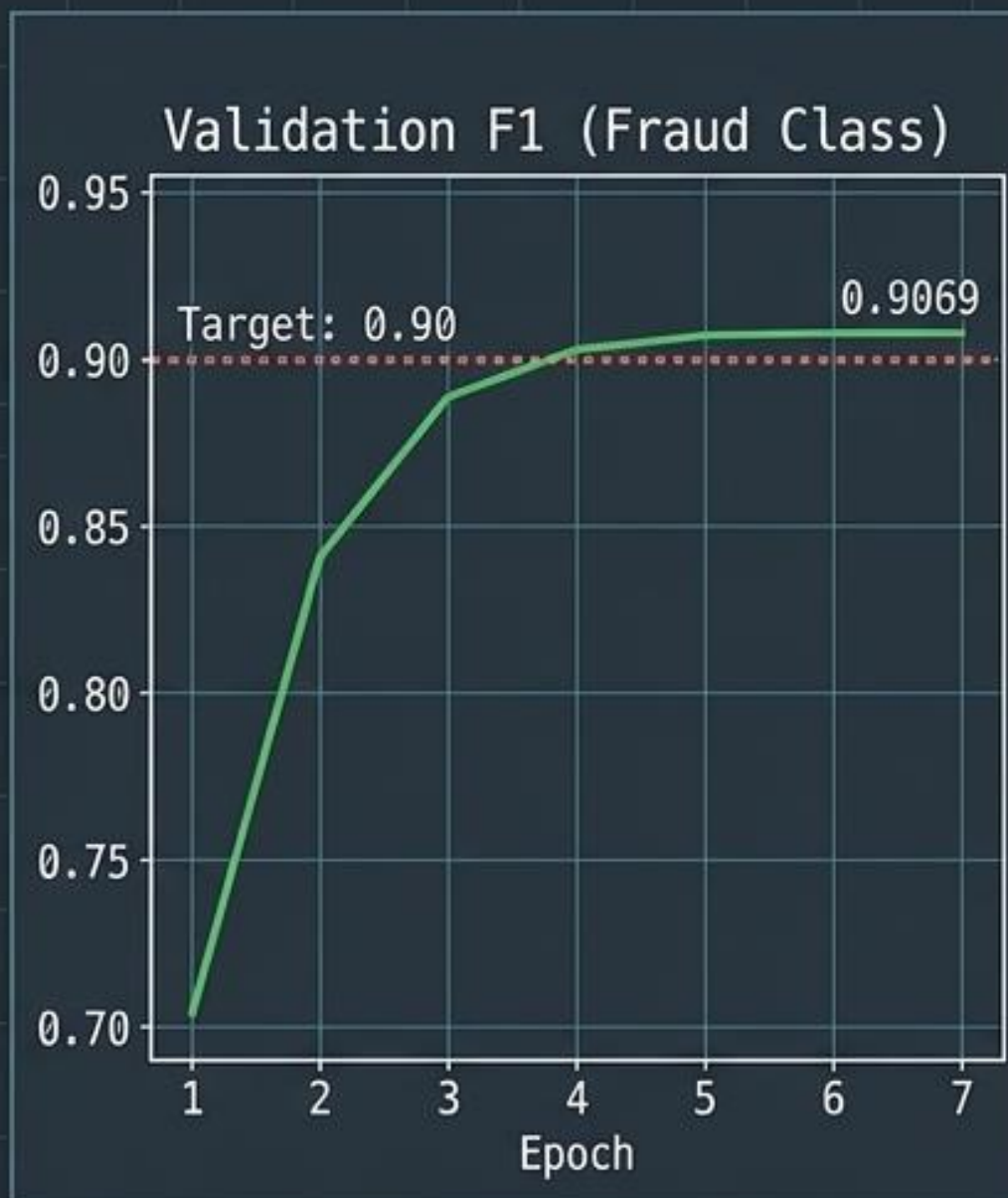
Trials are PRUNED unless
Recall ≥ 0.89 **AND** Precision ≥ 0.93
simultaneously.

This prevents optimizing F1 at the
expense of either metric.



System Evolution Map

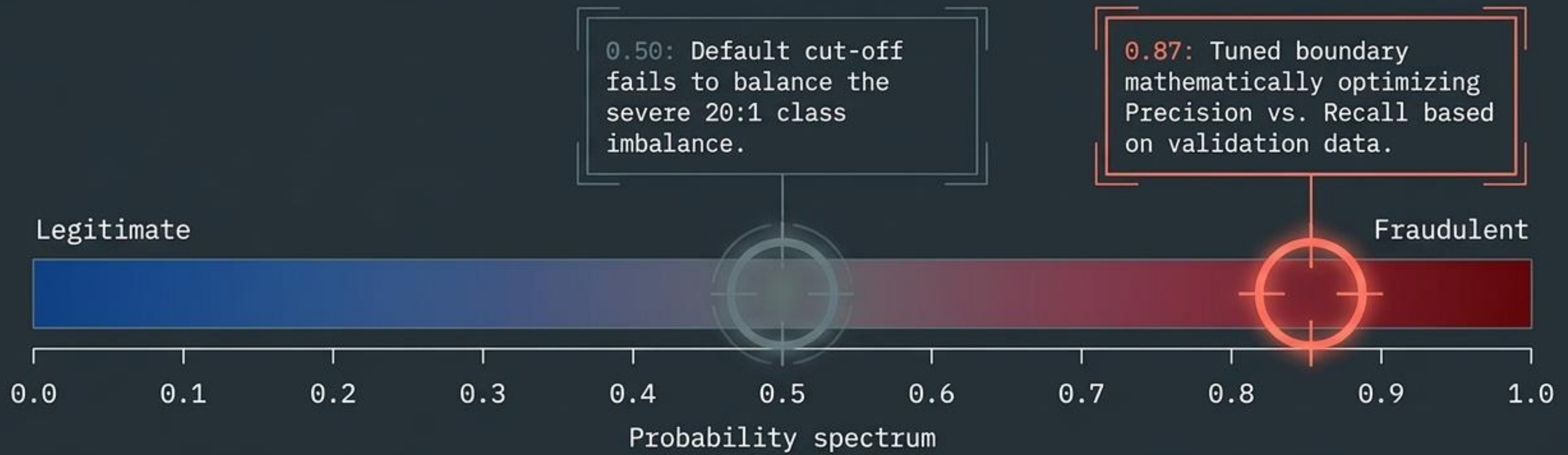




Diagnostic Insight

Automated Bayesian optimization (Optuna) successfully identified the exact learning rate and focal gamma values ($\gamma = 1.6920$) to stabilize the pipeline without manual guesswork, enabling stable convergence despite the severe 20:1 class imbalance.

Visualizing the Boundary: Threshold Tuning



Key Insight

The RoBERTa model does not output a simple Yes/No. It outputs a continuous probability spectrum. Moving the decision boundary to 0.87 forces the model to be highly confident before flagging a job, protecting precision.

TERM DICTIONARY

Probability Threshold: The exact numerical cutoff point where the model's prediction flips from Legitimate to Fraudulent.

Forensic Diagnostic Overlays

Threshold 0.50 (Default)

		Predicted Label	
		Legitimate	Fraudulent
True Label	Legitimate	2527	25
	Fraudulent	15	115

Threshold 0.82

		Predicted Label	
		Legitimate	Fraudulent
True Label	Legitimate	2544	8
	Fraudulent	18	112

Threshold 0.87 (Final)

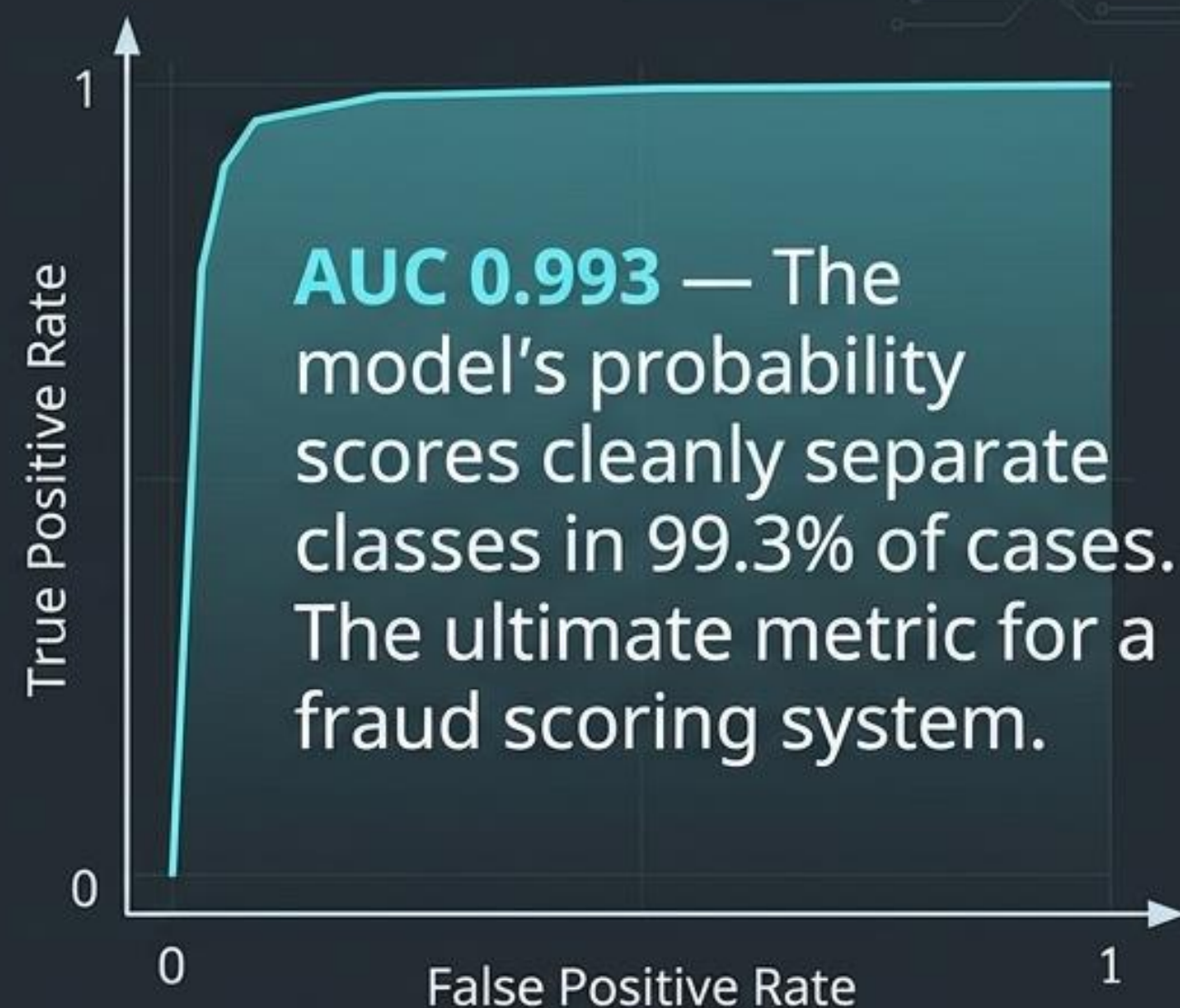
		Predicted Label	
		Legitimate	Fraudulent
True Label	Legitimate	2547	5
	Fraudulent	18	112

Diagnostic Insight

By raising the threshold to 0.87, we successfully crushed false alarms (dropping from 25 to 5), ensuring high platform trust, at the calculated expense of letting 3 additional fraudulent posts slip through.

Results — Near-Perfect AUC, High Precision

Metric	Target	Achieved	Status
F1 (fraud)	≥ 0.91	0.9069	Narrow miss (-0.3%)
Recall	≥ 0.89	0.8615	Miss (-2.9%)
Precision	≥ 0.93	0.9573	Met (+2.7%)
ROC-AUC	≥ 0.95	0.9930	Met (+4.3%)
MCC	—	0.8917	—



Threshold Calibration: Moving threshold from 0.50 → 0.87 added 3–4 F1 points at zero training cost.

Beyond Classification – 12-Tool Evidence Pipeline

Why a classifier isn't enough: It assigns probability, but it cannot verify external facts.



This produces human-readable evidence, not just a probability score.

Flask Web-App — Three Input Modes

Input Paths

1. **Paste text:**
Job description entered directly.
2. **File upload:**
PDF, DOCX, TXT, HTML, CSV.
3. **LinkedIn URL:**
Scrapes and analyzes live posting.

The screenshot shows a web application interface with a dark theme. On the left, there are three input sections: 'INPUT' (a large text area), 'File upload' (a button with an upload icon), and 'URL' (a text field with a link icon). On the right, there is a 'VERDICT' banner at the top, followed by a grid of 12 tool result cards, and a 'FULL LLM INVESTIGATION REPORT' section at the bottom. The interface is annotated with arrows from the 'Input Paths' list on the left, pointing to the corresponding input fields, and arrows from the 'What the User Sees' list on the right, pointing to the verdict banner, tool cards, and the full report section.

INPUT

File upload

URL

VERDICT: SUSPICIOUS (Confidence: 88%)

EMAIL_VERIFY Email Service, Score: 8/12	DOMAIN_REPUTATION Breitbart, 4/15 entries	WEBSITE_VERIFY No HTTPS, Score: 6/15	SCAM_SIGNALS Common keywords found, 5 hits
PHONE_CHECK Vop Service, Location: NY	JOB_BOARDS No active listings found	COMPANY_REGISTRY Firm found in major databases	SOCIAL_PROFILES Linked profiles: 2 profiles
WEBSITE_CONTENT Gutenberg, limited info	COMPANY_WIKIPEDIA No entry found	COMPANY_WEB_SEARCH Found results, poor relevance	COMPANY_OENS No public records found, 2...

FULL LLM INVESTIGATION REPORT

Full LLM investigation report, generated using GPT-4, based on the provided input. The report details the findings of the investigation, including the analysis of the provided text, file, and URL. The report is structured as follows:

1. **Input Analysis:** The input provided was a job description for a 'Software Engineer' position. The analysis identified several red flags, including the use of generic language, lack of specific details, and the presence of suspicious keywords.

2. **Tool Results:** The investigation utilized 12 tools to analyze the input. The results are summarized in the table above, showing various scores and findings across different categories such as email verification, domain reputation, website verification, scam signals, phone check, job boards, company registry, social profiles, website content, company Wikipedia, company web search, and company OENS.

3. **Verdict:** Based on the analysis and tool results, the overall verdict is **SUSPICIOUS** with a confidence score of 88%.

4. **Recommendations:** The report recommends further investigation, including contacting the company for more information, checking the company's website, and reviewing the job posting for any updates.

Full LLM investigation report generated using GPT-4, based on the provided input. The report details the findings of the investigation, including the analysis of the provided text, file, and URL. The report is structured as follows:

What the User Sees

- Verdict banner (SAFE / **SUSPICIOUS** / LIKELY_FAKE) with confidence score.
- 12 tool result cards, each with 2–4 sentence LLM summary.
- Full LLM-written investigation report.

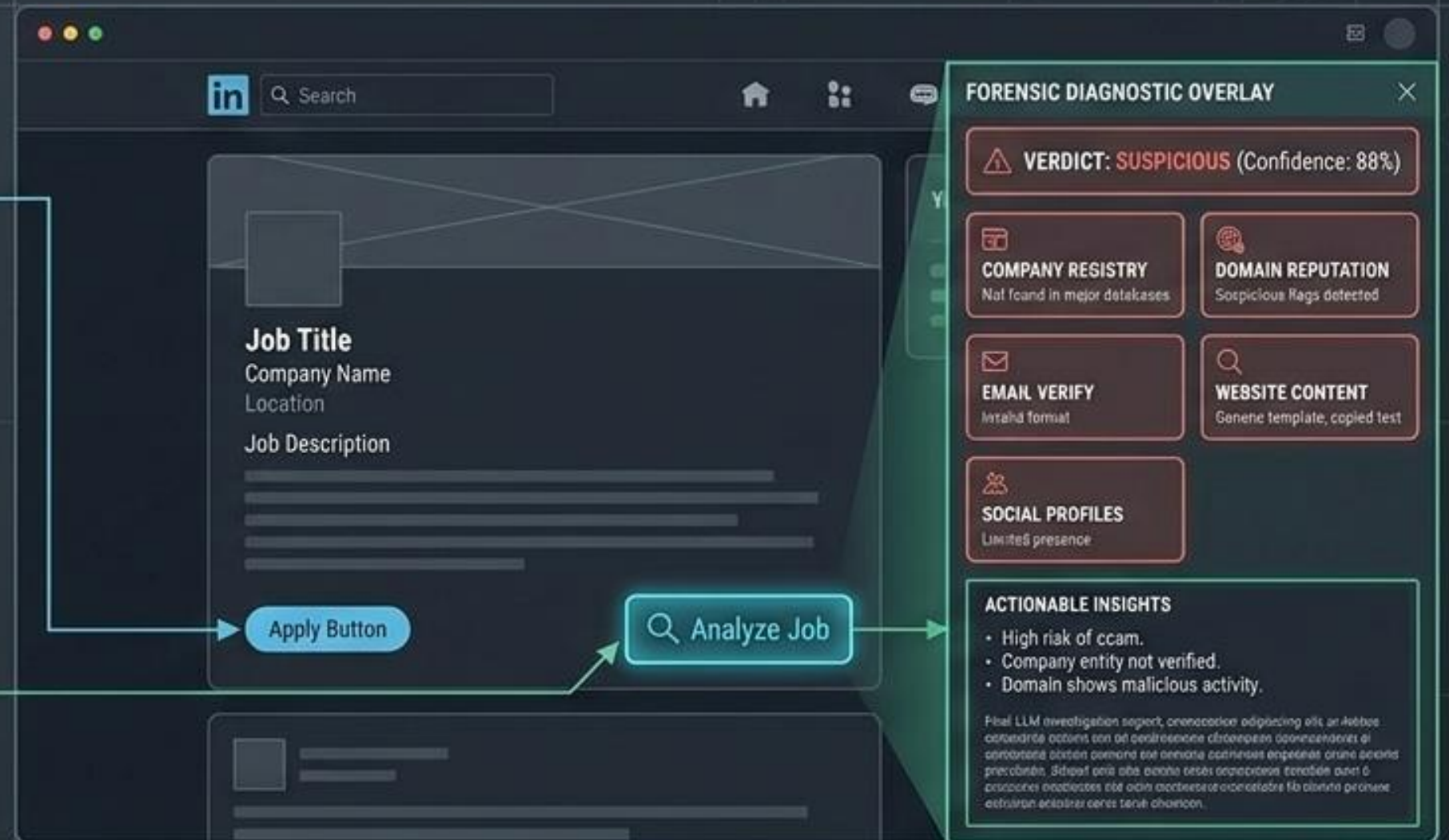
Real-Time Protection on LinkedIn

Bringing AI to the User

- Injects a floating “ **Analyze Job**” button directly into the LinkedIn DOM.
- Powered by Google Gemini API and Vanilla JS — zero server latency.

The Experience

- **1-Click Execution:** Scrapes active job details directly from the webpage.
- **Instant Verdict:** Generates a color-coded overlay panel within 3–5 seconds.
- **Actionable Insights:** Translates complex fraud signals into simple bullet points for the job seeker.



Team Contributions

Arun Dutta

- Led data preprocessing pipeline development.
- Synthesized literature review & gap analysis.
- Synthesized results for final reporting.

Hritik Roshan Maurya

- Engineered Agentic AI & LangChain orchestrator.
- Built 16-field extraction logic.
- Implemented Web-App REST API.

COLLABORATIVE
PROJECT
EXECUTION

Vivek Bajaj

- Directed ML pipeline strategy.
- Performed Optuna HPO (25 trials).
- Calibrated thresholds & deployed to HuggingFace.

Vishwas Mehta

- Developed Chrome Extension & Browser Integration.
- Implemented Metadata Anomaly Detector.
- Led LinkedIn DOM scraping efforts.